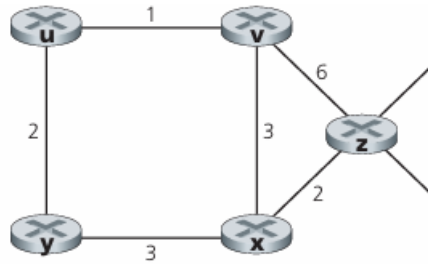


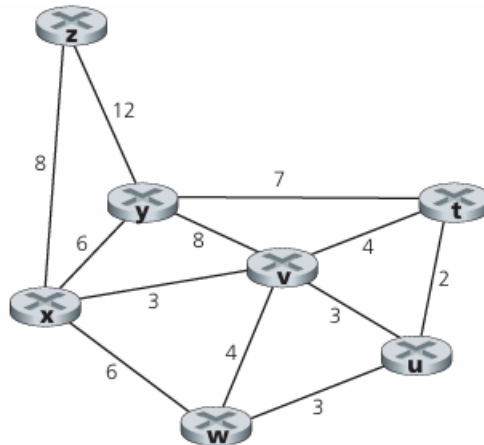
第五章作业

简答题

1. 考虑下图所示的网络，假设每个节点初始时都知晓其到所有相邻节点的开销。请基于**距离向量算法**，给出节点 **z** 的距离表。



2. 考虑下图所示的网络拓扑，请采用 Dijkstra 算法计算节点 t 到其他节点的最短路径，给出关键计算步骤即可。



3. 某自治系统 AS100 通过两个 ISP（AS200 和 AS300）连接到互联网。AS200 是 AS100 的提供商（provider），AS300 是 AS100 的对等节点（peer）。
 - 1) 当 AS100 收到来自 AS200 和 AS300 的同一目的网络的路由通告时，根据 BGP 路由选择规则，AS100 会优先选择哪条路径？为什么？
 - 2) 如果 AS100 希望将自身作为流量中转节点（transit AS），需要如何调整其 BGP 策略？可能存在哪些风险？

Lab5 基于 RIP 的路由配置实验

目标： 通过实验理解距离向量路由协议的基本原理，掌握简单路由配置。

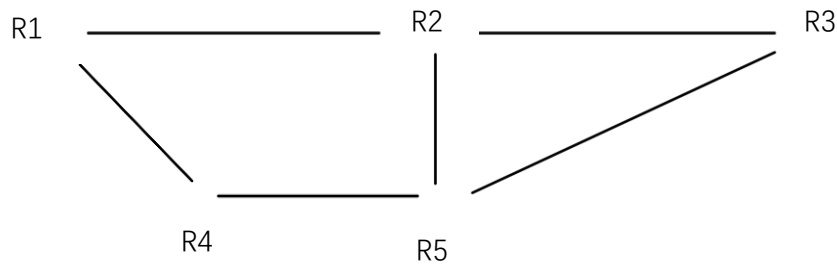
工具：

Mininet 或者 Docker 模拟网络拓扑，

支持 RIP 协议的软件：FRR

步骤：

1. 搭建如下拓扑：



所有链路初始代价设为 1.

2. 在每台路由器上配置 RIP 协议，确保路由收敛
3. 测试连通性：从 R1 ping R3 和 R5，记录结果
4. 断开 R2-R5 的链路，观察路由表更新
5. 重新连接 R2-R5，观察路由表更新

报告要求：

1. 提供各路由器的初始路由表和更新后的路由表截图。
2. 分析 RIP 协议在链路故障时的收敛时间及路由环路避免机制。
3. 回答：如果将所有链路代价改为非对称值（如 $R1-R2=1$ ， $R2-R1=5$ ），RIP 协议能否正常工作？为什么？

Instructions:

环境依赖: FRR, (docker or mininet)

```
Sudo apt install frr, docker
```

Mininet+FRR 示例:

1. 创建拓扑脚本 rip_lab.py

```
from mininet.net import Mininet
from mininet.node import Controller, OVSSwitch
from mininet.cli import CLI
from mininet.link import TCLink
from mininet.log import setLogLevel

def create_topology():
    net = Mininet(controller=Controller, switch=OVSSwitch, link=TCLink)

    # 添加 5 台路由器 (使用 Switch 模拟)
    r1 = net.addSwitch('r1')
    r2 = net.addSwitch('r2')
    r3 = net.addSwitch('r3')
    r4 = net.addSwitch('r4')
    r5 = net.addSwitch('r5')

    # 创建链路 (带宽=10Mbps, 延迟=1ms)
    net.addLink(r1, r2, bw=10, delay='1ms') # R1-R2
    net.addLink(r2, r3, bw=10, delay='1ms') # R2-R3
    net.addLink(r1, r4, bw=10, delay='1ms') # R1-R4
    net.addLink(r4, r5, bw=10, delay='1ms') # R4-R5
    net.addLink(r2, r5, bw=10, delay='1ms') # R2-R5
    net.addLink(r3, r5, bw=10, delay='1ms') # R3-R5

    # 启动网络
    net.start()

    # 配置 IP 地址 (每个接口一个子网)
    r1.cmd('ifconfig r1-eth0 10.0.1.1/24')
    r1.cmd('ifconfig r1-eth1 10.0.4.1/24')
```

```

r2.cmd('ifconfig r2-eth0 10.0.1.2/24')
r2.cmd('ifconfig r2-eth1 10.0.2.1/24')
r2.cmd('ifconfig r2-eth2 10.0.5.1/24')
r3.cmd('ifconfig r3-eth0 10.0.2.2/24')
r3.cmd('ifconfig r3-eth1 10.0.6.1/24')
r4.cmd('ifconfig r4-eth0 10.0.4.2/24')
r4.cmd('ifconfig r4-eth1 10.0.3.1/24')
r5.cmd('ifconfig r5-eth0 10.0.3.2/24')
r5.cmd('ifconfig r5-eth1 10.0.5.2/24')
r5.cmd('ifconfig r5-eth2 10.0.6.2/24')

# 启动 FRR 的 RIP 守护进程（每台路由器）
for router in [r1, r2, r3, r4, r5]:
    router.cmd('/usr/lib/frr/ripd -f /etc/frr/ripd.conf -i /tmp/ripd_{}.pid -d'.format(router.name))
    router.cmd('vtysh -b') # 进入 FRR 配置模式

# 进入命令行交互
CLI(net)
net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    create_topology()

```

2. 修改 FRR 配置文件：

```

! RIP configuration for R1
hostname R1
password zebra
!
router rip
version 2
network 10.0.1.0/24
network 10.0.4.0/24
!
log stdout
# 其他路由器的配置类似（修改 network 声明对应的子网）
# R2: network 10.0.1.0/24, 10.0.2.0/24, 10.0.5.0/24
# R3: network 10.0.2.0/24, 10.0.6.0/24
# R4: network 10.0.4.0/24, 10.0.3.0/24
# R5: network 10.0.3.0/24, 10.0.5.0/24, 10.0.6.0/24

```

3. 启动拓扑:

```
sudo python3 rip_lab.py
```

4. 在 Mininet CLI 中验证 RIP 路由表

```
mininet> r1 vtysh -c "show ip rip"
mininet> r2 vtysh -c "show ip route"
```

5. 测试连通性:

```
mininet> r1 ping -c3 10.0.6.1 # R1 ping R3 的接口
```

6. 模拟链路故障:

```
7. mininet> link r2 r5 down # 断开 R2-R5 链路
8. mininet> r2 vtysh -c "show ip rip" # 观察路由更新
```

注意事项:

1. 确保/etc/frr/目录下为每台路由器生成独立的配置文件。
2. 运行 Mininet 和 FRR 可能需要 **sudo** 权限

Docker+FRR 示例

1. 编写 Docker Compose 文件 docker-compose.yml

```
version: '3'

services:
  r1:
    image: frrouting/frr:latest
    container_name: r1
    command: /bin/sh -c "/usr/lib/frr/start-frr.sh && sleep infinity"
    cap_add:
      - NET_ADMIN
    volumes:
      - ./configs/r1:/etc/frr/
    networks:
```

```
net1:
  ipv4_address: 10.0.1.1
net4:
  ipv4_address: 10.0.4.1

r2:
  image: frrouting/frr:latest
  container_name: r2
  command: /bin/sh -c "/usr/lib/frr/start-frr.sh && sleep infinity"
  cap_add:
    - NET_ADMIN
  volumes:
    - ./configs/r2:/etc/frr/
  networks:
    net1:
      ipv4_address: 10.0.1.2
    net2:
      ipv4_address: 10.0.2.1
    net5:
      ipv4_address: 10.0.5.1

r3:
  image: frrouting/frr:latest
  container_name: r3
  command: /bin/sh -c "/usr/lib/frr/start-frr.sh && sleep infinity"
  cap_add:
    - NET_ADMIN
  volumes:
    - ./configs/r3:/etc/frr/
  networks:
    net2:
      ipv4_address: 10.0.2.2
    net6:
      ipv4_address: 10.0.6.1

r4:
  image: frrouting/frr:latest
  container_name: r4
  command: /bin/sh -c "/usr/lib/frr/start-frr.sh && sleep infinity"
  cap_add:
    - NET_ADMIN
  volumes:
    - ./configs/r4:/etc/frr/
  networks:
```

```
net4:
  ipv4_address: 10.0.4.2
net3:
  ipv4_address: 10.0.3.1

r5:
  image: frrouting/frr:latest
  container_name: r5
  command: /bin/sh -c "/usr/lib/frr/start-frr.sh && sleep infinity"
  cap_add:
    - NET_ADMIN
  volumes:
    - ./configs/r5:/etc/frr/
  networks:
    net3:
      ipv4_address: 10.0.3.2
    net5:
      ipv4_address: 10.0.5.2
    net6:
      ipv4_address: 10.0.6.2

networks:
  net1:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.1.0/24
  net2:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.2.0/24
  net3:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.3.0/24
  net4:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.4.0/24
  net5:
    driver: bridge
```

```
ipam:
  config:
    - subnet: 10.0.5.0/24
net6:
  driver: bridge
  ipam:
    config:
      - subnet: 10.0.6.0/24
```

2. 创建 FRR 配置文件目录和文件

```
mkdir -p configs/{r1,r2,r3,r4,r5}

# 示例: R1 的配置文件 configs/r1/ripd.conf
cat <<EOF > configs/r1/ripd.conf
hostname R1
password zebra
!
router rip
  version 2
  network 10.0.1.0/24
  network 10.0.4.0/24
!
log stdout
EOF

# 其他路由器的配置类似（修改 network 声明对应的子网）
# R2: network 10.0.1.0/24, 10.0.2.0/24, 10.0.5.0/24
# R3: network 10.0.2.0/24, 10.0.6.0/24
# R4: network 10.0.4.0/24, 10.0.3.0/24
# R5: network 10.0.3.0/24, 10.0.5.0/24, 10.0.6.0/24
```

3. 启动容器

```
docker-compose up -d
```

4. 进入路由器命令行（R1 为例子）

```
docker exec -it r1 vtysh
```

5. 查看 RIP 路由表

```
R1# show ip rip
R1# show ip route
```


6. 测试连通性

```
# 在 R1 中 ping R3 的接口 (10.0.6.1)
docker exec r1 ping -c 3 10.0.6.1
```

7. 模拟故障

```
# 断开 R2 和 R5 之间的链路（在宿主机操作）
docker network disconnect rip-lab_net5 r2
# 观察路由表更新
docker exec -it r2 vtysh -c "show ip rip"
```

8. 恢复链路

```
docker network connect rip-lab_net5 r2 --ip 10.0.5.1
```

注意事项：

1. 如遇容器启动失败：检查配置文件路径和权限，确保 configs 目录正确挂载。
2. 如遇网络不通：使用 `docker network inspect <network_name>` 检查容器 IP 是否分配正确。
3. 如遇 RIP 未收敛：确认所有路由器的 `ripd.conf` 中宣告了直连子网。